Java Operators - Complete In-Depth Explanation

Let me break down every concept from this transcript thoroughly.

---

1. ARITHMETIC OPERATORS

Basic Operators (+, -, *, /, %)

These are the simplest operators — same as math.

```
int a = 5;
int b = 10;

int c = a + b;  // 15
int d = a - b;  // -5
int e = a * b;  // 50
int f = b / a;  // 2
int g = b % a;  // 0 (remainder of 10÷5)
```

Modulus operator (%) — very important concept:

- It divides and gives you the **remainder**, not the quotient
- Example: `10 % 3 = 1` (because 10 = 3×3 + **1**)
- Example: `10 % 5 = 0` (because 10 = 5×2 + **0**)

Compound Assignment Operators (+=, -=, *=, /=, %=)

Instead of writing `h = h + 2`, Java gives you a shortcut:

```
int h = 7;

h += 2;   // same as h = h + 2  → h becomes 9
h -= 2;   // same as h = h - 2  → h becomes 7
h *= 3;   // same as h = h * 3  → h becomes 21
```

```
h /= 5;   // same as h = h / 5  → h becomes 4
h %= 3;   // same as h = h % 3  → h becomes 1
```

Why use these? They're shorter, cleaner, and slightly faster. Real-world code almost always uses `+=` style over `h = h + 2`.

Increment & Decrement Operators (++, --)

```
int i = 6;
i++;   // i becomes 7  (same as i = i + 1, or i += 1)
i--;   // i becomes 6  (same as i = i - 1, or i -= 1)
```

Why do these exist? Because in programming (especially loops, arrays), you constantly need to move forward/backward by exactly 1. `i++` is the cleanest way.

Note: `**` and `//` do NOT exist as operators. Only `++` and `--` exist because incrementing/decrementing by 1 is extremely common.

PRE vs POST Increment — The Most Important Detail

This is where most beginners get confused.

```
int j = 7;

// POST-increment
int k = j++;
// k = 7 (old value assigned FIRST, then j increments)
// After this line: k=7, j=8

// PRE-increment
int l = ++j;
// j increments FIRST, then new value assigned to l
// After this line: l=9, j=9
```

Mental model:

- `j++` → "Give me j's current value, THEN increase j"
- `++j` → "Increase j FIRST, THEN give me j's new value"

Real example to burn this in:

```
int x = 5;
System.out.println(x++);  // prints 5, then x becomes 6
System.out.println(++x);  // x becomes 7, then prints 7
```

Why does this matter? In loops, `for(int i=0; i<n; i++)` vs `for(int i=0; i<n; ++i)` — the behavior differs when the result is being assigned or used somewhere.

---

2. RELATIONAL OPERATORS

These operators **compare** two values and always return a **boolean** (true/false).

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 10`	false
`!=`	Not equal to	`5 != 10`	true
`<`	Less than	`5 < 10`	true
`>`	Greater than	`5 > 10`	false
`<=`	Less than or equal	`10 <= 10`	true
`>=`	Greater than or equal	`5 >= 10`	false

```
int a = 5, b = 10;

boolean c = (a == b);   // false
boolean d = (a != b);   // true
boolean e = (a < b);    // true
boolean f = (a > b);    // false
boolean g = (a <= b);   // true
boolean h = (a >= b);   // false
```

Critical difference — `=` vs `==`:

```
a = b;    // ASSIGNMENT — puts b's value INTO a. a is now 10
```

```
a == b;   // COMPARISON — asks "is a equal to b?" Returns true/false
```

This is one of the most common bugs beginners write — using `=` when they meant `==`.

---

3. BITWISE OPERATORS

Understanding Binary First

Everything in memory is stored as 0s and 1s. Before bitwise operators make sense, you need to understand binary:

```
Decimal 2  →  Binary: 0000 0010
Decimal 3  →  Binary: 0000 0011
Decimal 8  →  Binary: 0000 1000
```

Each position represents a power of 2:

```
Position:  128  64  32  16  8  4  2  1
Value 8:    0    0   0   0  1  0  0  0
Value 3:    0    0   0   0  0  0  1  1
```

Truth Table — The Foundation

A	B	A & B	A | B	A ^ B	~A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

AND operator `&`

Rule: Both bits must be 1 for result to be 1. If even one is 0, result is 0.

```
2  →  0000 0010
```

```
& 3  →  0000 0011
──────────────────
  2  →  0000 0010
```

Step by step: Compare each bit position:

- Position 1: 1 AND 1 = **1**
- Position 2: 1 AND 0 = **0** ← wait, let me redo this properly

```
2 = 0 0 0 0 0 0 1 0
3 = 0 0 0 0 0 0 1 1
  ─────────────────
&   0 0 0 0 0 0 1 0  = 2
```

Result = **2**

OR operator |

Rule: If at least ONE bit is 1, result is 1. Only 0 OR 0 = 0.

```
2 = 0 0 0 0 0 0 1 0
3 = 0 0 0 0 0 0 1 1
  ─────────────────
|   0 0 0 0 0 0 1 1  = 3
```

Result = **3**

XOR (Exclusive OR) operator ^

Rule: Result is 1 only when there's an **ODD number of 1s**. Same bits = 0, different bits = 1.

```
2 = 0 0 0 0 0 0 1 0
3 = 0 0 0 0 0 0 1 1
  ─────────────────
^   0 0 0 0 0 0 0 1  = 1
```

Result = **1**

Clever XOR trick: XOR of a number with itself is always 0. Used in interview questions to find the unique element in an array.

NOT operator `~`

Rule: Flips every single bit. 0→1, 1→0. Unary operator (works on one operand).

```
int a = 2;
// 2 in 32-bit: 0000 0000 0000 0000 0000 0000 0000 0010
// ~2:          1111 1111 1111 1111 1111 1111 1111 1101
```

Why does `~2 = -3`? Because of **Two's Complement** (how Java stores negative numbers).

Formula: `~n = -(n+1)` always. So `~2 = -3`, `~5 = -6`, etc.

```
System.out.println(~2);   // -3
System.out.println(~10);  // -11
```

Full code verification:

```
int a = 2, b = 3;
int c = a & b;  // 2
int d = a | b;  // 3
int e = a ^ b;  // 1
int f = ~a;     // -3

System.out.println(c + " " + d + " " + e + " " + f);
// Output: 2 3 1 -3
```

---

4. BITWISE SHIFT OPERATORS

This is the most complex topic in the video. Let's go very deep.

Left Shift `<<`

Syntax: `number << n` = shift all bits n positions to the LEFT

```
byte b = 8;
// 8 in binary: 0000 1000

b = (byte)(b << 1);
// After shift:  0001 0000  = 16
```

Visual:

```
Before: 0 0 0 0 1 0 0 0   (= 8)
After:  0 0 0 1 0 0 0 0   (= 16)
            ↑ bit moved left by 1 position
```

Pattern: Left shift by 1 = multiply by 2

```
8  << 1 = 16   (8 × 2)
16 << 1 = 32   (16 × 2)
32 << 1 = 64   (32 × 2)
```

General rule: `n << k = n × 2^k`

TYPE PROMOTION — Critical Hidden Behavior

This is what most tutorials miss:

Java NEVER performs shift operations on byte or short directly.

When you write `b << 1` where b is a byte, Java secretly does this:

1. Converts `b` from 8-bit byte → 32-bit int
2. Performs the shift on the 32-bit int
3. Returns a 32-bit int result

This is why you MUST cast back:

```
byte b = 8;
b = b << 1;        // COMPILER ERROR! Can't assign int to byte
b = (byte)(b << 1); // CORRECT — explicit cast back to byte
```

What happens when you shift beyond the range?

Let's trace `byte b = 1` being left-shifted repeatedly:

```
Start:  0000 0001  = 1
<< 1:   0000 0010  = 2
<< 1:   0000 0100  = 4
<< 1:   0000 1000  = 8
<< 1:   0001 0000  = 16
<< 1:   0010 0000  = 32
<< 1:   0100 0000  = 64
<< 1:   1000 0000  = ???
```

When the 1 reaches the MSB (Most Significant Bit = leftmost bit), that means NEGATIVE in Java!

Why? Java uses **Two's Complement** where the MSB is the sign bit:

- MSB = 0 → positive number
- MSB = 1 → negative number

So `1000 0000` = **-128** (the minimum byte value)

Shift one more time:

```
<< 1:   0000 0000  = 0
```

The 1 falls out of the 8-bit range → result is 0.

BUT WAIT — This is the TRUNCATED view. Actual computation happens in 32-bit:

```
byte b = 1 promoted to int:
0000 0000 0000 0000 0000 0000 0000 0001

After b << 8 (32-bit):
0000 0000 0000 0000 0000 0001 0000 0000  = 256

When cast back to byte (only keep last 8 bits):
0000 0000  = 0
```

So when you see 0 in the byte, the 32-bit int value is actually **256**, **512**, etc. — they just get truncated.

The Modulo 32 Rule for int

For `int` (32-bit), Java applies: **shift amount = shift amount % 32**

```
int i = 1;
i << 32   // same as i << 0  (32 % 32 = 0) → returns 1!
i << 33   // same as i << 1  (33 % 32 = 1) → returns 2
i << 64   // same as i << 0  (64 % 32 = 0) → returns 1!
```

Why? Because shifting an int more than 31 positions is meaningless — the bit would fall off the edge. So Java caps it with modulo 32.

For `long` (64-bit): shift amount = shift amount % 64

Right Shift `>>`

Arithmetic Right Shift — shifts bits RIGHT, but PRESERVES the sign bit.

```
int b = 16;
// 0001 0000

b >> 1;
// 0000 1000  = 8
```

Pattern: Right shift by 1 = divide by 2

```
64 >> 1 = 32
32 >> 1 = 16
16 >> 1 = 8
```

For negative numbers — sign is preserved:

-128 in binary: 1000 0000

-128 >> 1: 1100 0000 = -64

Notice: a new **1** was inserted on the left (because MSB was 1 = negative). This preserves the sign.

This is called **Sign Extension** — the incoming bit matches the original sign bit.

Unsigned Right Shift `>>>`

Logical Right Shift — shifts bits RIGHT, always inserts **0** on the left regardless of sign.

```
int b = -128;
// 1000 0000 (8-bit representation)

b >>> 1;
// 0100 0000  = +64  ← negative became positive!
```

`>>` would give `-64` (preserves sign) `>>>` gives `+64` (ignores sign, always puts 0)

Use case: Image processing, pixel values — you never want negative pixel values, so you use `>>>` to ensure you always get positive numbers when shifting.

Complete Shift Comparison

```
int a = -8;
System.out.println(a >> 1);   // -4  (sign preserved)
System.out.println(a >>> 1);  // 2147483644  (huge positive number!)
```

---

5. LOGICAL OPERATORS

`&&` (Logical AND) and `||` (Logical OR)

Unlike bitwise operators that work on **bits**, logical operators work on **boolean expressions**.

```
int a = 5, b = 10, c = 15;

boolean result1 = (a < b) && (b < c);  // true AND true = true
boolean result2 = (a > b) && (b < c);  // false AND true = false
boolean result3 = (a > b) || (b < c);  // false OR true = true
```

Same truth table as bitwise, just with true/false instead of 1/0.

SHORT-CIRCUIT EVALUATION — Very Important!

This is what makes && and || special (and what makes them different from & and |).

Short-circuit with &&:

```
boolean result = (a > b) && (b < c);
//             ^^^^^^^^^
//             This is FALSE
// Java STOPS HERE. Never evaluates (b < c).
// Why? Because false AND anything = false. No need to check further.
```

Short-circuit with ||:

```
boolean result = (a < b) || (b < c);
//             ^^^^^^^^^
//             This is TRUE
// Java STOPS HERE. Never evaluates (b < c).
// Why? Because true OR anything = true. No need to check further.
```

Real-world importance — Null checks:

```
// Without short-circuit, this could crash:
if (obj != null & obj.getName().equals("John"))  // NullPointerException if obj is null!

// With short-circuit, this is safe:
if (obj != null && obj.getName().equals("John"))  // Safe! If obj is null, second part never runs.
```

Single & vs Double && — Interview Trap Question!

```
// Both give same final result, but behave differently internally:

boolean r1 = (a < b) && (b < c);  // Short-circuits. Second part may NOT evaluate.
boolean r2 = (a < b) &  (b < c);  // NO short-circuit. Second part ALWAYS evaluates.
```

When to use single & with expressions? Almost never in real code. It's mainly an interview concept. But knowing it distinguishes someone who truly understands operators.

---

6. ASSIGNMENT OPERATOR

Basic Assignment =

```
int a = 5;  // Store value 5 in variable a
```

Critical distinction:

```
a = b;   // ASSIGNMENT: copy b's value into a
a == b;  // COMPARISON: check if a equals b (returns boolean)
```

Assignment Chaining

```
int a, b, c;
a = b = c = 10;
// Evaluates RIGHT TO LEFT:
// First: c = 10
// Then:  b = 10 (b = c)
// Then:  a = 10 (a = b)
// Result: a=10, b=10, c=10
```

---

7. OPERATOR PRECEDENCE

When multiple operators appear in one expression, which one runs first?

Precedence Table (Highest to Lowest)

Priority	Operators
1 (Highest)	( ) brackets
2	++ -- postfix
3	++ -- prefix, ~, ! (unary)
4	*, /, %
5	+, -

6	`<<`, `>>`, `>>>`
7	`<`, `>`, `<=`, `>=`
8	`==`, `!=`
9	`&` (bitwise AND)
10	`^` (bitwise XOR)
11	`|` (bitwise OR)
12	`&&` (logical AND)
13	`||` (logical OR)
14 (Lowest)	`=`, `+=`, `-=`, etc.

Practical Example

```
int result = 2 + 3 * 4;
// * has higher precedence than +
// So: 3*4=12 first, then 2+12=14
// result = 14

int result = (2 + 3) * 4;
// Brackets force + to happen first
// So: 2+3=5 first, then 5*4=20
// result = 20
```

The Real Advice — Use Brackets!

Nobody memorizes the full precedence table. In real code, always use brackets for complex expressions:

```
// Hard to read, depends on memorizing precedence:
int x = a + b * c - d / e & f << 2;

// Clear and readable:
int x = ((a + (b * c) - (d / e)) & (f << 2));
```

Key things to actually remember:

1. `*`, `/`, `%` before `+`, `-` (BODMAS)
2. Assignment `=` is always LAST
3. Brackets `( )` always have HIGHEST priority
4. When operators have same precedence → left to right (except `=` which goes right to left)

Summary Mind Map

```
OPERATORS IN JAVA
│
├── ARITHMETIC  (+, -, *, /, %)
│   ├── Compound (+=, -=, *=, /=, %=)
│   └── Increment/Decrement (++, --)
│       ├── Postfix: x++ (use then increment)
│       └── Prefix:  ++x (increment then use)
│
├── RELATIONAL  (==, !=, <, >, <=, >=)
│   └── Always returns boolean
│
├── BITWISE  (&, |, ^, ~)
│   ├── Works on individual bits
│   └── Uses Truth Tables
│
├── SHIFT  (<<, >>, >>>)
│   ├── << Left shift = multiply by 2^n
│   ├── >> Right shift = divide by 2^n (sign preserved)
│   ├── >>> Unsigned right shift (sign NOT preserved)
│   └── Type promotion: always operates on int/long
│
├── LOGICAL  (&&, ||)
│   ├── Works on boolean expressions
│   └── SHORT-CIRCUIT: stops early when result is known
│       ├── && stops if first is FALSE
│       └── || stops if first is TRUE
│
├── ASSIGNMENT  (=)
│   └── Lowest precedence, evaluates right to left
│
└── PRECEDENCE
    └── Use brackets () to control order explicitly
```

The most important takeaways are the **pre/post increment difference**, **short-circuit evaluation**, the **type promotion in bitwise shifts**, and the **= vs == distinction** — these come up constantly in both real code and interviews.